SQL Analytic Functions & Data Integration

SQL ANALYTIC FUNCTIONS

Core Syntax Pattern

```
FUNCTION() OVER ([PARTITION BY col] ORDER BY col [WINDOW])
```

- Evaluated **after** GROUP BY and HAVING
- Returns one value per row (not aggregated like GROUP BY)

LESSON 1-2: RANKING FUNCTIONS

Four Ranking Functions

Function	Behavior	Use Case
`RANK()`	Gaps with ties (1,2,2,4)	Standard rankings
`DENSE_RANK()`	No gaps (1,2,2,3)	Continuous rankings
`ROW_NUMBER()`	Always unique (1,2,3,4)	Total ordering needed
`NTILE(n)`	Divides into n buckets	Quartiles, percentiles

Key Concepts

- **PARTITION BY**: Creates independent ranking groups (like ranking sales by state)
- **ORDER BY**: Determines ranking criteria
- Processing order: FROM → WHERE → GROUP BY → HAVING → **Analytic** → SELECT → ORDER BY

LESSON 3-4: WINDOW AGGREGATES

Window Types

Physical (ROWS): Count-based

- `ROWS UNBOUNDED PRECEDING` - cumulative from start
- `ROWS 2 PRECEDING` - last 2 rows
- `ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING` - centered 3-row window

Logical (RANGE): Value-based

- `RANGE BETWEEN 1 PRECEDING AND 1 FOLLOWING` - based on ORDER BY column values
- `RANGE INTERVAL '1' YEAR PRECEDING` - time-based windows

Common Patterns

```
-- Cumulative sum
SUM(SalesDollar) OVER (ORDER BY Date ROWS UNBOUNDED PRECEDIN
G)

-- Moving average (3-row centered)
AVG(SalesDollar) OVER (ORDER BY Date ROWS BETWEEN 1 PRECEDING
AND 1 FOLLOWING)

-- Partitioned cumulative
SUM(SalesDollar) OVER (PARTITION BY Brand ORDER BY Month ROWS
UNBOUNDED PRECEDING)
```

LESSON 5: RATIO COMPARISON FUNCTIONS

Three Key Functions

1. RATIO_TO_REPORT - Contribution ratios

- Shows "part of whole" (always sum to 1)
- Example: Each store's % of total sales

```
RATIO_TO_REPORT(SUM(Sales)) OVER (PARTITION BY Year)
```

2. PERCENT_RANK - Distribution position

- Formula: `(rank - 1) / (N - 1)`
- Range: **0 to 1** (inclusive)
- Best for: Comparing ranks across different-sized groups

3. CUME_DIST - Cumulative distribution

- Formula: `rows_up_to_and_including / total_rows`
- Range: **>0 to 1**
- Best for: Finding top X% performers

```
-- Top 30% by price
WHERE CUME_DIST() OVER (ORDER BY Price DESC) <= 0.3
```

DATA INTEGRATION CONCEPTS

LESSON 1: Refresh Processing

Two Main Activities:

1. **Initial Load** - One-time, open-ended, requires data profiling
2. **Refresh** - Ongoing, scheduled, incremental updates

Key Constraints:

- **Window constraints**: Limited time for processing
- **Capacity constraints**: Hardware/network limitations
- **Freshness requirements**: How current must data be?

Workflow: Extract → Clean → Transform → Load (into warehouse)

LESSON 2: Change Data Types

Type	Source	SQL Access	Data Quality
Cooperative	Triggers on source DB	Yes	Good
Logged	Application logs	Limited	Variable
Queryable	Event tables with timestamps	Yes	Good
Snapshot	File comparisons (deltas)	No	Poor

Common Data Quality Issues:

- Multiple identifiers for same entity
- Different units/formats
- Missing/conflicting values
- Different update timestamps

LESSON 3: Data Cleaning Tasks

1. PARSING - Separate elements from raw text

- Uses: Regular expressions, NLP
- Example: Split "John A. Doe, VP Sales" into FirstName, MI, LastName, Title

2. CORRECTING - Fix missing/conflicting values

- Missing: Use defaults, averages, or predictive models
- Conflicts: Choose most recent or credible source

3. STANDARDIZING - Apply consistent formats

- Unit conversions (kg→lbs)
- Abbreviations (WA for Washington)
- Business rules (standardized job titles)